

Software Requirements Specification

A Mobile Application for Facility Management

Team

[Team name]

Members

Gamila Mohamed | 13007300 (Team
Lead)

Malak Madyan | 13006076

Habiba Ahmed | 13007391

Miriam Ramy | 13007122

Jana Kassem | 13001189

Tutorial Details

3 | Moamen Atia

1. Introduction

This project introduces a mobile-based facility management system intended to help with the effective maintenance and preservation of university campus assets. The app offers a digital solution that allows members of the university community to submit infrastructure, cleanliness, sustainability, and maintenance concerns using an efficient and user-friendly mobile app. By digitizing the reporting and handling process, the system aims to enhance communication, shorten response times, and ensure a safe, clean, and well-maintained campus environment. Furthermore, the application encourages community engagement and fosters a sense of shared responsibility among campus members.

The system is designed for several university user groups, each with specific responsibilities and permissions. Faculty members and students can report problems and check their status, while facility managers are in charge of analyzing filed complaints, allocating tasks to maintenance staff, and tracking progress. Maintenance staff receive job assignments and offer progress updates after addressing issues. An administrator role is to manage users, permissions, and general system operations, resulting in a secure and structured workflow.

The software's scope involves the development of a mobile application. Core functions include issue submission with photos, descriptions, and location details, issue categorization, role-based access control, task assignment, alerts, and real-time status updates. The system functions as a centralized platform, connecting the university community with the Facility Management team through a defined problem lifecycle, from submission to resolution. This closed-loop strategy improves transparency, accountability, and collaboration, eventually benefiting the campus's sustainability, safety, and productivity.

1.1 Product Vision & Scope

The product envisions a digitally united campus in which facility related issues are identified and resolved with minimum delay and maximum clarity. It aims to improve how campus related problems are addressed and managed by replacing informal or fragmented systems with a centralized and efficient mobile solution. The system intends to enhance proactive maintenance practices, boost operational responsiveness, and contribute to a cohesive, resilient, and sustainable campus environment by utilizing structured reporting and coordinated task management. The long-term goal is to maintain a feeling of responsibility and collaboration among all campus stakeholders.

The application is a mobile solution that addresses campus facility problems. Users may submit issue descriptions, while authorized workers manage task assignments, track progress, and confirm resolution to maintain regulated and organized workflows. The system's key functions include issue categorization, location-based reporting, worker assignment, status updates, and notifications.

1.2 Definitions and Acronyms

1. Ticket:

A digital maintenance request submitted through the mobile application to report a facility related issue.

2. Reporter:

A student or staff member who submits a maintenance ticket using the system.

3. Facility Manager:

A user responsible for reviewing submitted tickets, assigning tasks to workers, and monitoring issue resolution.

4. Worker:

A maintenance staff member who receives assigned tasks and updates the status after resolving issues.

5. Admin:

A system administrator responsible for managing user accounts, roles, and overall system control.

6. RBAC (Role-Based Access Control):

A security mechanism that restricts system access based on predefined user roles.

7. Dashboard:

An interface that provides an overview of maintenance tickets and their current status.

8. Notification:

A real-time alert informing users about updates or status changes related to tickets.

9. Workflow:

The sequence of steps from ticket submission to issue resolution.

10. Status:

The current state of a ticket (e.g., Pending, Assigned, In Progress, Resolved).

2. Functional Requirements

Describes what the system should do, first in the form of user stories then as a list of functional requirements for each user/role. A UML use-case diagrams should then be included to link specific features and requirements.

2.1 User stories:

Community Members, Facility Managers, Workers and Admins.

Community Member

1. As a Community Member, I want to register and log in to the system, so that I can securely access my account.
2. As a Community Member, I want to log out so that my account remains secure on shared devices.
3. As a Community Member, I want to select my role during registration, so that the system assigns the correct access level.
4. As a Community Member, I want my account to be approved by the Facility Manager before activation, so that only authorized users can access the system.
5. As a Community Member, I want to select a category for the issue so that it is properly classified.
6. As a Community Member, I want to upload a photo of the issue, so that the maintenance team can visually understand the problem.
7. As a Community Member, I want to add a description of the issue, so that the maintenance team clearly understands what needs to be fixed.
8. As a Community Member, I want to manually enter the building and room number when submitting a ticket, so that the assigned worker can easily locate the issue.
9. As a Community Member, I want to receive a confirmation push notification after submitting a ticket, so that I know my request has been successfully recorded.
10. As a Community Member, I want to view all my submitted tickets, so that I can check their status.
11. As a Community Member, I want to track the status of my submitted tickets, so that I know whether they are submitted, assigned, in progress, denied, or completed.
12. As a Community Member, I want to receive push notifications when the ticket status changes, so that I stay informed without checking the system repeatedly.

Worker

1. As a Worker, I want to select the worker role during registration, so that I receive tasks relevant to my responsibilities.
2. As a Worker, I want to log in using my email and password so that I can access the system.
3. As a Worker, I want to log out so that my data remains secure on shared devices.
4. As a Worker, I want to receive push notifications when I am assigned a task, so that I am immediately aware of new responsibilities.
5. As a Worker, I want to view the details of my assigned tickets, including description, photo, and location, so that I can perform the task properly.

6. As a Worker, I want to update the task status, so that the system reflects accurate progress.
7. As a Worker, I want to upload a photo after completing the task, so that proof of work is documented.
8. As a Worker, I want to add notes or a description of the completed work, so that the Facility Manager can review what was done.

Facility Manager

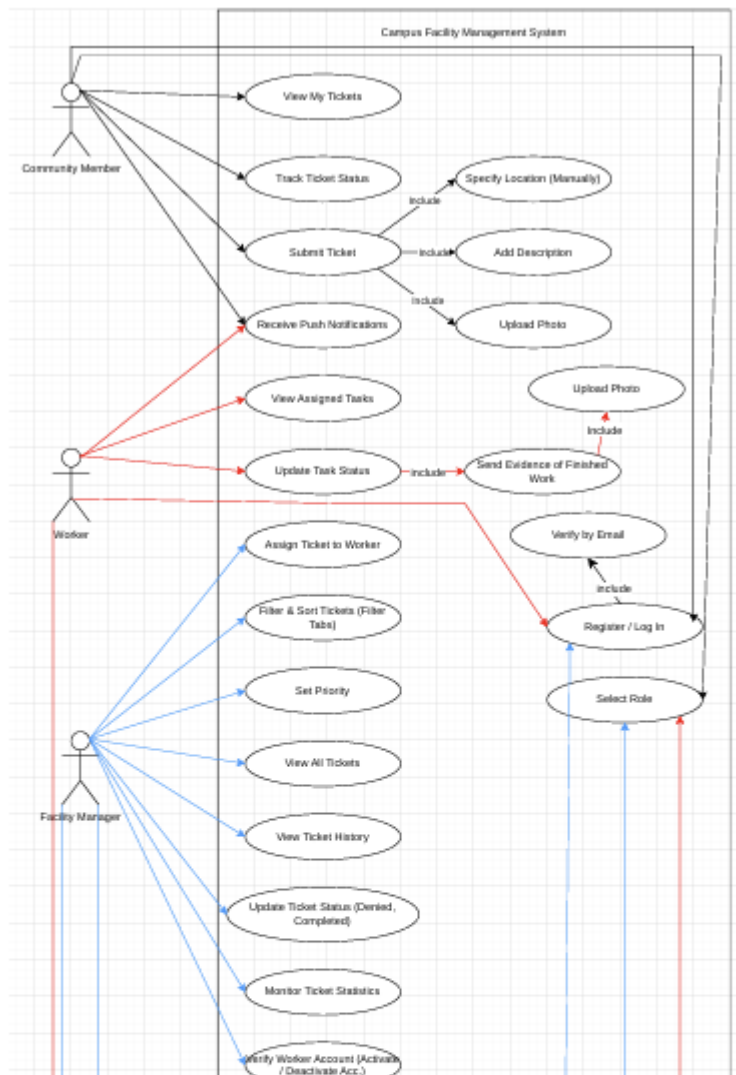
1. As a Facility Manager, I want to log in using my email and password so that I can access the system.
2. As a Facility Manager, I want to log out so that my data remains secure on shared devices.
3. As a Facility Manager, I want to approve or reject newly registered accounts, so that only authorized users can access the system.
4. As a Facility Manager, I want to verify and activate or deactivate worker accounts, so that system security is maintained.
5. As a Facility Manager, I want to assign roles and permissions to users, so that access control is properly managed.
6. As a Facility Manager, I want to view all submitted tickets in a centralized dashboard, so that I can monitor maintenance activities.
7. As a Facility Manager, I want to filter and sort tickets using status-based tabs, so that I can efficiently manage the ticket lifecycle.
8. As a Facility Manager, I want to assign tickets to specific workers, so that responsibilities are clearly defined.
9. As a Facility Manager, I want to set priority levels for tickets, so that critical issues are handled first.
10. As a Facility Manager, I want to update the ticket status (Submitted, Assigned, In Progress, Denied, Completed), so that the system reflects real-time progress.
11. As a Facility Manager, I want to view the history of each ticket, so that I can ensure accountability and proper follow-up.
12. As a Facility Manager, I want to monitor ticket statistics and system performance, so that I can evaluate overall efficiency.

System Admin

1. As an Admin, I want to log in so that I can access the system.
2. As an Admin, I want to log out so that my data remains secure.

3. As an Admin, I want to activate or deactivate user accounts so that system rules are enforced.
4. As an Admin, I want to view all users so that I can manage the system.

2.2 UML Use-case Diagram



3. Non-functional Requirements:

Response Time:

The system must ensure that all page load times and API responses are completed in under three 3 seconds under normal operating conditions. This includes loading dashboards, viewing issue details, updating statuses, and retrieving assigned tasks. Issue submission, including image upload, description entry, and location selection, should be processed efficiently and completed within a few seconds on a stable internet connection. Maintaining fast response times is critical to ensuring a smooth and frustration-free user experience, encouraging community members and staff to use the application regularly without delays.

Concurrent Users:

The application must support at least 500 simultaneous users without noticeable performance degradation. This includes users submitting issues, facility managers assigning tasks, and workers updating statuses at the same time. The backend system should be designed to efficiently manage multiple concurrent API requests, database transactions, and image uploads. Proper server configuration, load management, and scalable architecture must be implemented to ensure system stability during peak academic hours or high-demand periods.

API Security:

The system must implement strong API security measures to protect against unauthorized access and malicious attacks. Rate limiting must be applied to prevent abuse, spam submissions, or denial-of-service attempts. All input data must be validated and sanitized before processing to prevent SQL injection, cross-site scripting (XSS), and other common vulnerabilities. Authentication and role-based authorization must be enforced to ensure that users can only access features and data permitted by their role (e.g., community member, worker, facility manager, admin). All communication between the mobile application and backend must be encrypted using HTTPS.

Backup & Recovery:

Automated daily backups of the PostgreSQL database must be performed to protect all stored data, including user accounts, issue records, and status updates. Backup files should be securely stored and periodically tested to ensure successful restoration when needed. A disaster recovery plan must be documented and implemented to restore the system within an acceptable timeframe in case of hardware failure, cyberattack, or data corruption. These measures ensure business continuity and minimize data loss risks.

Usability:

The mobile application must provide an intuitive, simple, and user-friendly interface that allows community members to submit tickets in under one minute. The issue reporting process should require minimal steps: capturing or uploading a photo, writing a short description, and selecting the location. Navigation should be clear, buttons should be properly labeled, and status updates should be easy to understand (e.g., Submitted, Assigned, In Progress, Resolved). The design should follow consistent layout patterns to reduce confusion and ensure that users of different technical skill levels can comfortably use the system.

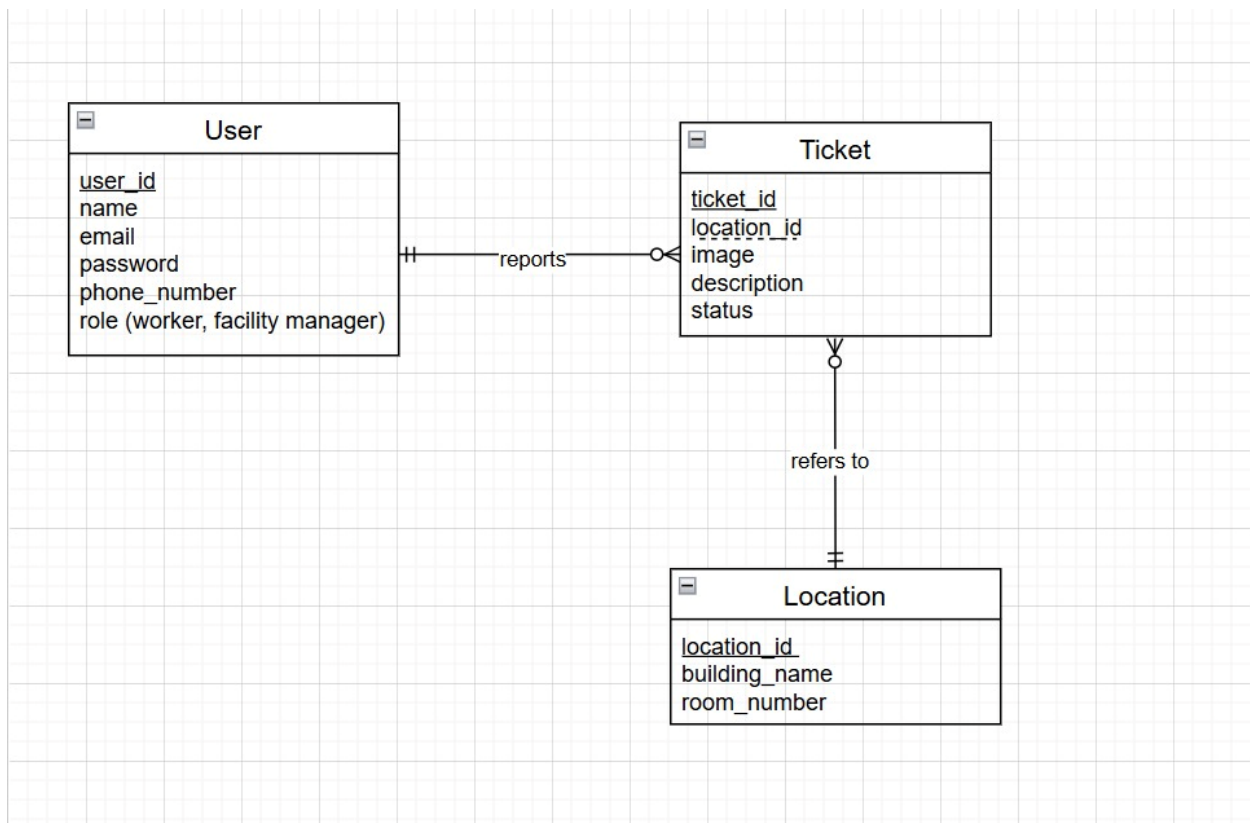
Reliability:

The system must operate consistently without crashes, errors, or data loss. It must handle image uploads robustly, even when network connections are unstable, by providing retry mechanisms and clear error messages if a submission fails. All issue updates and status changes must be accurately stored in the database and reflected immediately for authorized users. Logging and monitoring mechanisms should be implemented to detect system failures or unexpected behavior, ensuring that problems are quickly identified and resolved.

4. Constraints

- The application must support image upload for tickets.
- The system is initially designed for use within GIU campus only.

5. Entity Relation Diagram



6.

Technology Stack

- Response Time: Page load times under three seconds
- Concurrent Users: Support for at least 500 simultaneous users
- API Security: Rate limiting and input validation
- Backup & Recovery: Automated daily backups with disaster recovery plan